

Leçon 3 : Les structures itératives en Langage Pascal

AYIKPA KACOUTCHY JEAN : Enseignant -
Chercheur

Table des matières



I - Introduction	3
II - 1- La structure for .. do	4
III - Application 1 :	6
IV - 2- La structure while .. do	8
V - Application 2 :	10
VI - 3- La structure repeat .. until	12
VII - Application 3 :	13
VIII - Solutions des exercices	15

Introduction



Les structures itératives, encore appelées structures répétitives font partie des structures de contrôle en programmation et en langage pascal en l'occurrence. Elles permettent d'exécuter plusieurs fois un même bloc d'instructions en tenant compte d'un critère d'exécution. Ainsi, il existe trois types de structures itératives en langage pascal que sont :

- La structure for .. do
- La structure while .. do
- La structure repeat .. until

1- La structure for .. do



Définition : 1.1- Définition

La boucle « *for* » permet de répéter une instruction un nombre donné de fois jusqu'à un maximum. Elle se caractérise par le fait que l'on doit connaître à l'avance le nombre d'itérations que l'on va devoir effectuer.

Syntaxe : 1.2- Syntaxe

for variation := départ *to* arrivée *do*

begin

instruction;

end;

Remarque :

- *variation* : il s'agit de la variable qui recevra les différentes valeurs de la boucle
- *départ* : il s'agit de la valeur de départ de la boucle
- *arrivée* : il s'agit de la dernière valeur de la boucle

Exemple : 1.3- Application

Énoncé :

Écrire un programme qui affiche les 10 premiers entiers positifs.

Solution :

```
program nombreentiers ;
```

```
uses crt ;
```

```
var
```

```
i : integer ;
```

```
begin
```

```
for i := 1 to 20 do
```

```
begin
```

```
writeln('i =',i) ;
```

```
end ;
```

```
end.
```

Explication : On obtient le résultat comme suit :

```
i = 1
```

```
i = 2
```

```
i = 3
```

```
i = 4
```

1- La structure for .. do

i = 5

i = 6

i = 7

i = 8

i = 9

i = 10

Application 1 :



Exercice 1

[Solution n°1 p 15]

Donner la syntaxe correcte :

- ☐ for variation := arrivée to départ do
begin
instruction;
end;
- ☐ for variation := départ to arrivée do
begin
instruction;
end;
- ☐ for variation := départ to arrivée do
instruction;
end;

Exercice 2

[Solution n°2 p 15]

Énoncé :

Écrire un programme qui permet de saisir un nombre entier , le programme devra permettre de faire la somme de nombre impairs et le total des nombres impairs.

TAF : Veuillez remplir toutes les trous avec les valeurs qui conviennent.

NB : Toutes les réponses ne doivent pas avoir des espaces et elles doivent être en minuscule .

Solution :

```
calculnbreimpair
```

```
uses crt;
```

```
N, Impair, Somme, i :
```

```
Entrez votre nombre :
```

```
Impair
```

```
Somme
```

```
for i:=1
```

```
begin
```

```
Somme
```

```
Impair
```

2- La structure while .. do



Définition : 2.1- Définition

La boucle « *while* » permet d'effectuer des itérations sous certaines conditions, avant d'effectuer une itération la boucle vérifie si la condition est vrai.

Syntaxe : 2.2- Syntaxe

Initialisation;

while condition *do*

begin

instructions ;

variation;

end;

Remarque :

- L'initialisation consiste à attribuer une valeur de départ à une variable (elle peut se faire par affectation ou par saisie).
- La condition est celle qui permet si elle est vrai de faire une itération.
- La variation consiste à aller à la valeur suivante de la variable initialisée
- La boucle « *while* » sans variation devient une boucle infinie

Exemple : 2.3- Application

Énoncé :

Écrire un programme qui affiche les 10 premiers entiers positifs.

Solution :

program nombreentiers ;

uses crt ;

var

i : integer ;

begin

i := 1 ; // *Initialisation*

while i <= 10 do

begin

writeln('i =',i) ;

i := i+1 ; // *Variation*

end ;

end.

Explication : On obtient le résultat comme suit :

i = 1
i = 2
i = 3
i = 4
i = 5
i = 6
i = 7
i = 8
i = 9
i = 10

Application 2 :


IV

Exercice 1

[Solution n°3 p 16]

Donner la syntaxe correcte :

- ☐ initialisation ;
while condition do
begin
instructions ;
variation;
end;
- ☐ initialisation ;
while condition do
begin
instructions ;
end;
- ☐ initialisation ;
while condition do
instructions ;
variation;
end;

Exercice 2

[Solution n°4 p 16]

Énoncé :

Écrire un programme qui vérifie si un mot saisi est un palindrome. Un palindrome est un mot qui se lit dans les deux sens (de gauche vers droite et de la droite vers gauche). exemple : php, serres, etc

TAF : Veuillez remplir toutes les trous avec les valeurs qui conviennent.

NB :

- Toutes les réponses ne doivent pas avoir des espaces et elles doivent être en minuscule.
- length : ramène la taille d'une chaîne(nombre de caractère de la chaîne).

Solution :

```

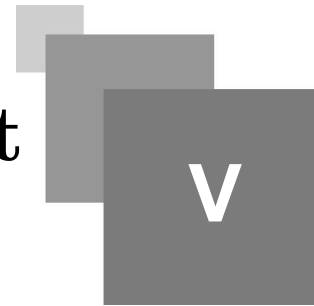
program palindrome ;
uses crt ;
var
mot : 
i,j : 
begin
writeln('Entrer un mot :');

i := 1;
j := length(mot);
while ((  ()) do
begin
i := 
j := 
end;
if () then
begin
 mot, ' est un palindrome');

else
begin
 mot, ' n'est pas un palindrome' ) ;
end
end.

```

3- La structure repeat .. until



Définition : 3.1- Définition

La boucle « *repeat* » permet d'effectuer au moins une itération avant de vérifier la condition et elle continue l'itération jusqu'à ce que la condition soit vraie avant de sortir de la boucle.

Syntaxe : 3.2- Syntaxe

```
repeat
instruction ;
until condition ;
```

Remarque :

- La boucle « *repeat* » commence par le mot réservé *repeat* et s'achève par un *until* .
- L'instruction est exécutée au moins une fois.
- Ici il n'y a pas de begin et end pour le *repeat*.

Exemple : 3.3- Exemple

Énoncé :

Écrire un programme qui permet de saisir un nombre entier positif et ceux jusqu'à ce que la condition soit vérifiée.

Solution :

```
program nombrepositif ;
uses crt ;
var
Nbre : entier ;
begin
repeat
writeln('Entrer un nombre entier positif') ;
readln(Nbre) ;
until Nbre >0 ;
end.
```

Explication :

Cette boucle enverra toujours le message « Entrer un nombre entier positif » jusqu'à ce que l'utilisateur entre un nombre respectant la condition.

Application 3 :

VI

Exercice 1

[Solution n°5 p 17]

Donner la syntaxe correcte :

- ☐ repeat
instruction ;
until condition ;
- ☐ repeat
begin
instruction ;
end ;
until condition ;
- ☐ repeat
instruction ;
do condition ;

Exercice 2

[Solution n°6 p 17]

Énoncé :

Écrire un programme qui oblige l'utilisateur a saisi un nombre positif, ensuite le programme doit dit si le nombre saisi est pair.

TAF : Veuillez remplir toutes les trous avec les valeurs qui conviennent.

NB :

- Toutes les réponses ne doivent pas avoir des espaces et elles doivent être en minuscule.

Solution :

```
program nombrepositif;
```

```
uses crt;
```

```
var
```

```
nbre : integer ;
```

```


```

```


```

```
writeln('Entrer un nombre positif :') ;
```

```


```

```


```

```


```

```
begin
```

```
writeln('Le nombre saisi est un nombre pair');
```

```


```

```


```

Solutions des exercices

> Solution n°1

Exercice p. 6

- ☐ for variation := arrivée to départ do
begin
instruction;
end;
- ☒ for variation := départ to arrivée do
begin
instruction;
end;
- ☐ for variation := départ to arrivée do
instruction;
end;

> Solution n° 2

Exercice p. 7

Énoncé :

Écrire un programme qui permet de saisir un nombre entier , le programme devra permettre de faire la somme de nombre impairs et le total des nombres impairs.

TAF : Veuillez remplir toutes les trous avec les valeurs qui conviennent.

NB : Toutes les réponses ne doivent pas avoir des espaces et elles doivent être en minuscule .

Solution :

```

program calculnbreimpair ;
uses crt;
var
N, Impair, Somme, i : integer;
begin
writeln(' Entrez votre nombre : ');
readln(N);
Impair :=0;
Somme :=0;
for i:=1 to N do
begin
if(imod2=1)then

```

```

begin
Somme :=Somme+i;
Impair :=Impair+1;
end;
end;
end.

```

> Solution n°3

Exercice p. 10

- ☒ initialisation ;
while condition do
begin
instructions ;
variation;
end;
- ☐ initialisation ;
while condition do
begin
instructions ;
end;
- ☐ initialisation ;
while condition do
instructions ;
variation;
end;

> Solution n° 4

Exercice p. 11

Énoncé :

Écrire un programme qui vérifie si un mot saisi est un palindrome. Un palindrome est un mot qui se lit dans les deux sens (de gauche vers droite et de la droite vers gauche). exemple : php, serres, etc

TAF : Veuillez remplir toutes les trous avec les valeurs qui conviennent.

NB :

- Toutes les réponses ne doivent pas avoir des espaces et elles doivent être en minuscule.
- length : ramène la taille d'une chaîne(nombre de caractère de la chaîne).

Solution :

```

program palindrome ;
uses crt ;
var
mot :string;
i,j :integer;
begin

```



```

writeln('Entrer un mot :');
readln(mot);
i :=1;
j :=length(mot);
while (( i<j ) and ( mot[i]=mot[j] )) do
begin
i := i+1;
j := j-1;
end;
if ( mot[i]=mot[j] ) then
begin
writeln( mot,' est un palindrome');
end
else
begin
writeln( mot,' n''est pas un palindrome' ) ;
end
end.

```

> Solution n°5

Exercice p. 13

- ☒ repeat
 - instruction ;
 - until condition ;
- ☐ repeat
 - begin
 - instruction ;
 - end ;
 - until condition ;
- ☐ repeat
 - instruction ;
 - do condition ;

> Solution n° 6

Exercice p. 14

Énoncé :

Écrire un programme qui oblige l'utilisateur a saisi un nombre positif, ensuite le programme doit dit si le nombre saisi est pair.

TAF : Veuillez remplir toutes les trous avec les valeurs qui conviennent.

NB :

- Toutes les réponses ne doivent pas avoir des espaces et elles doivent être en minuscule.

Solution :

```

program nombrepositif;

```

```
uses crt;  
var  
  nbre : integer ;  
begin  
  repeat  
    writeln('Entrer un nombre positif :)') ;  
    readln(nbre);  
  until  nbre>0;  
  if(nbre mod 2=0) then  
  begin  
    writeln('Le nombre saisi est un nombre pair');  
  end;  
end.
```